Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S09: Hands-On Lab Walkthrough & CLI Guide

Spark SQL, Advanced Joins & Distribution Optimization

1. Lab Objectives

In this hands-on engineering lab, you will apply advanced join optimization techniques in PySpark:
1. Ingest cleaned Nile Retail sales data from Parquet storage (produced in S08) and join it with the dimension table `data/customers.csv`.
2. Protect downstream pipelines against join fan-out anomalies through strict primary key uniqueness assertions and window function deduplication.
3. Disassemble physical execution plans via `explain(mode="formatted")` and verify zero-shuffle execution on the large fact table using Broadcast Hash Join (BHJ).
4. Benchmark performance and resource utilization between Broadcast Hash Join and Sort-Merge Join (SMJ).
5. Execute automated test suites via `pytest` to validate data pipeline correctness.

2. Environment Setup & Prerequisites

Open your terminal in the lab root directory and activate the verified virtual environment with Java 17 LTS:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab

# Activate Python virtual environment
source .venv/bin/activate

# Configure Java 17 LTS paths
if [ -d "${HOME}/.local/share/jvm/jdk-17.0.21+9" ]; then
  export JAVA_HOME="${HOME}/.local/share/jvm/jdk-17.0.21+9"
fi
export PATH="←!—KATEX_INLINE_0⟶{PATH}"
export PYSPARK_PYTHON="$(pwd)/.venv/bin/python"
export PYSPARK_DRIVER_PYTHON="$(pwd)/.venv/bin/python"

python -c "import pyspark; print('PySpark Version:', pyspark.__version__)"
```

Expected Output:

```
PySpark Version: 3.5.9
```

---

3. Step 1: Customer Dimension Exploration & Deduplication

Inspect the customer dimension dataset `data/customers.csv`. In real-world enterprise databases, customer records often contain duplicate entries due to address updates or multiple sync jobs:

```
head -n 10 data/customers.csv
```

To eliminate join multiplication (fan-out) without dropping valid customer data, we apply a Window function partitioned by `customer_id` and ordered by `updated_at DESC`:

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

window_spec = Window.partitionBy("customer_id").orderBy(F.col("updated_at").desc())
clean_customers_df = (
    raw_customers_df
    .withColumn("row_num", F.row_number().over(window_spec))
    .filter(F.col("row_num") == 1)
    .drop("row_num")
)
```

---

4. Step 2: Safe Join Execution via `python/s09_join_check.py`

Review the production join script `python/s09_join_check.py`. It implements two critical enterprise defensive design principles:

1. **Pre-Join Assertion:** Enforce that the dimension table has zero duplicate keys prior to joining:

```
python
   assert clean_customers_df.count() ==
clean_customers_df.select("customer_id").distinct().count(), "Duplicate keys detected in
dimension table!"
```

2. **Explicit Join Hinting:** Force Broadcast Hash Join when the dimension table fits comfortably in executor memory:

```
python
   joined_df = orders_df.join(
       F.broadcast(clean_customers_df),
       on="customer_id",
       how="inner"
   )
```

Execute the join check script:

```
python python/s09_join_check.py
```

Expected Output:

```
[INFO] Loaded 1,000,000 order fact records from Parquet.
[INFO] Loaded 10,000 raw customer records.
[INFO] Deduplicated customer dimension: 9,850 unique customers.
[INFO] Executing Broadcast Hash Join (BHJ)...
[INFO] Join verification successful: Row count matched expected 985,000 inner matches.
[INFO] Physical Plan verified: Zero shuffle exchange operators on fact table.
```

---

5. Step 3: Physical Plan Disassembly (BHJ vs SMJ)

Inspect the formatted physical plan to verify the Catalyst optimizer's execution strategy:

```
python -c '
from python.s09_join_check import build_joined_pipeline, create_spark
spark = create_spark()
joined_df = build_joined_pipeline(spark)
joined_df.explain(mode="formatted")
'
```

Look for the key physical plan operators:
- `BroadcastExchange`: Driver serializes and broadcasts the small table to all executors.
- `BroadcastHashJoin [customer_id#...], [customer_id#...], Inner, BuildRight`: In-memory hash lookup on the worker nodes.
- Notice the absence of `Exchange hashpartitioning` on the orders table.

---

6. Step 4: Automated Testing with Pytest

Run the session verification test suite:

```
pytest tests/test_s09_joins.py -v
```

Expected Output:

```
tests/test_s09_joins.py::test_deduplication_removes_duplicate_customers PASSED [ 33%]
tests/test_s09_joins.py::test_broadcast_join_prevents_row_multiplication PASSED [ 66%]
tests/test_s09_joins.py::test_physical_plan_contains_broadcast_operator PASSED  [100%]
============================== 3 passed in 4.12s ===============================
```

7. Step 5: Master Verification Script

Run the session verification script to record automated proof:

```
bash Trainer/S09_05_verify_s09.sh
```

Expected Output:

```
[PASS] S09 Join Optimization and Data Skew Defenses fully verified.
```

🌟 Gold Stretch Challenge: Key Salting for Extreme Data Skew

Distinguished Engineer Challenge: If you complete the core lab before time expires, implement a production-grade Key Salting pipeline to resolve skew:
1. Simulate a severe data skew scenario where 80% of orders belong to `customer_id = 9999` (e.g., a massive wholesale institutional buyer).
2. Attempt a standard Sort-Merge Join (`spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)`) and observe one executor task hanging (straggler task).
3. Implement 2-way key salting:
- On the skewed fact table: `salted_df = df.withColumn("salt", (F.rand() * 10).cast("int")).withColumn("salted_key", F.concat_ws("_", "customer_id", "salt"))`
- On the replicated dimension table: Explode the dimension table with an array of salts `[0..9]` and generate matching `salted_key` .
4. Execute the join on `salted_key` and confirm that all 10 executor partitions finish in parallel with uniform execution duration.